

RabbitMQ

1. What is RabbitMQ?

RabbitMQ is a message broker: a server that accepts messages from producers and delivers them to consumers.

Property	Detail
Speaks	AMQP 0-9-1, and MQTT and STOMP through plugins
Written in	Erlang, which is why clustering is built in
Routing lives	In the broker, not in the consumer

RabbitMQ is a smart broker with simple consumers. The decision about which consumer receives a message is made by the broker, from rules the consumer declared once. That is the design choice everything else follows from.

2. RabbitMQ vs Pub/Sub

The two solve the same problem and divide the work differently. The clearest way in is that the words map onto each other:

RabbitMQ	Pub/Sub	Is
Exchange	Topic	Where a producer publishes
Queue	Subscription	Where messages wait for one consumer
Binding and routing key	Subscription filter	What decides who receives it
Prefetch	Flow control	How much one consumer holds at once
Dead-letter exchange	Dead-letter topic	Where a failing message ends up

A RabbitMQ queue and a Pub/Sub subscription are the same idea. Both are where messages wait, and in both the count decides the pattern: one shared means consumers compete, one each means everybody gets a copy.

Where they differ is what each is good at:

	RabbitMQ	Pub/Sub
Routing	Four exchange types and wildcards	A topic, and an attribute filter
Operations	A cluster to run, or pay someone to	None. It is a service

	RabbitMQ	Pub/Sub
Scale	High, with tuning	Effectively unlimited, untuned
Messages wait	Until acknowledged, subject to queue limits	Up to the retention period
Ordering	One consumer, or a consistent-hash exchange	Ordering keys
Protocol	AMQP, an open standard	Google's API

Choose RabbitMQ when the routing is the interesting part. Several consumers wanting different slices of one stream, decided by rules on the server, is what exchanges and bindings are for.

Choose Pub/Sub when the volume or the operations are the interesting part. There is no cluster to size, no disk alarm to watch, and no upper bound worth planning for.

3. Why use RabbitMQ?

Problem	What RabbitMQ does
Work must not hold up the request	Queue it and answer the caller
A burst arrives faster than it can be handled	The queue absorbs it
Work has to survive a consumer restart	Messages wait in a durable queue
Several services need the same event	A fanout exchange, one queue each
Work must be spread across workers	One queue, many consumers competing
Different messages go to different places	A topic exchange and routing keys

Routing is what RabbitMQ is for. Other brokers move messages; this one decides where they go, using rules held on the server rather than in each consumer.

4. Message broker

A message broker sits between applications, taking messages from one and delivering them to others. Neither side knows the other.

Without a broker	With one
The caller must know the receiver's address	It knows an exchange name
The receiver must be up	The message waits
A burst reaches the receiver directly	The queue absorbs it
Adding a receiver changes the caller	Add a queue and a binding

What the broker adds is a place for a message to wait. Everything else, including routing, follows from having somewhere to keep it.

5. Producer

A producer publishes a message to an exchange. It never publishes to a queue.

```
rabbitTemplate.convertAndSend("orders", "order.placed", event);
```

The three arguments are the whole model: which exchange, what routing key, and the message.

Publishing to a queue is not possible, and looks possible. The default exchange is named `""` and routes to the queue whose name matches the routing key, so `convertAndSend("", "billing.queue", event)` appears to address a queue directly. It is still an exchange doing the work.

A publish succeeds even when nothing receives it. By default a message that matches no binding is silently discarded. Publisher confirms tell the producer the broker accepted the message, and the `mandatory` flag has an unroutable message returned instead of dropped.

6. Consumer

A consumer takes messages from a queue and processes them.

```
@RabbitListener(queues = "billing.queue")
public void handle(OrderPlaced event) {
    charge(event);
}
```

Messages are pushed to it by the broker. Pulling one at a time is possible and is slow enough that it is only for scripts.

Prefetch is the setting that matters. It caps how many unacknowledged messages one consumer may hold at once.

```
spring.rabbitmq.listener.simple.prefetch=20
```

An unlimited prefetch is the classic mistake. The broker hands the whole queue to the first consumer that connects, so a second consumer starts with nothing to do and the work is not shared at all. A small number keeps the queue spread across the consumers that are actually free.

7. Message and its properties

A message is a body of bytes plus properties the broker and the consumer can read.

Property	Holds
<code>content_type</code>	How to interpret the body, such as <code>application/json</code>
<code>delivery_mode</code>	1 for transient, 2 for persistent
<code>correlation_id</code>	Ties a reply to its request
<code>reply_to</code>	The queue a reply should go to
<code>headers</code>	Anything else, and what a headers exchange matches on

Persistence needs two things, and one alone does nothing. The queue must be declared durable so it survives a restart, and the message must be published with `delivery_mode: 2` so it is written to disk. A persistent message in a transient queue is lost with the queue.

8. Queue

A queue is a named buffer holding messages until a consumer takes them. It is the only thing in RabbitMQ that stores anything.

Setting	Does
<code>durable</code>	Survives a broker restart
<code>exclusive</code>	Belongs to one connection, deleted when it closes
<code>auto-delete</code>	Deleted when the last consumer disconnects
<code>x-message-ttl</code>	Messages expire after a time
<code>x-max-length</code>	The queue drops or dead-letters beyond a size
<code>x-dead-letter-exchange</code>	Where rejected and expired messages go

Exchanges hold nothing. A message published when no queue is bound has nowhere to wait, so it is discarded. The queue must exist before the message is published, not after.

9. Queue lifecycle

A queue is created by declaring it, and either side may do so.

Stage	What happens
Declared	Created if absent. Doing it again is harmless
In use	Holds messages, with consumers attached
Deleted	Explicitly, or by <code>auto-delete</code> , <code>exclusive</code> or <code>x-expires</code>

Declaring the same queue with different arguments fails. A second declare with a different TTL or dead-letter exchange is rejected as `PRECONDITION_FAILED` and closes the channel, which is why changing a queue's settings means deleting and recreating it.

Inside the queue, each message moves through its own states:

```
published → ready → delivered (unacked) → acked, and removed
                                     → nacked, and requeued or dead-lettered
                                     → expired by TTL
```

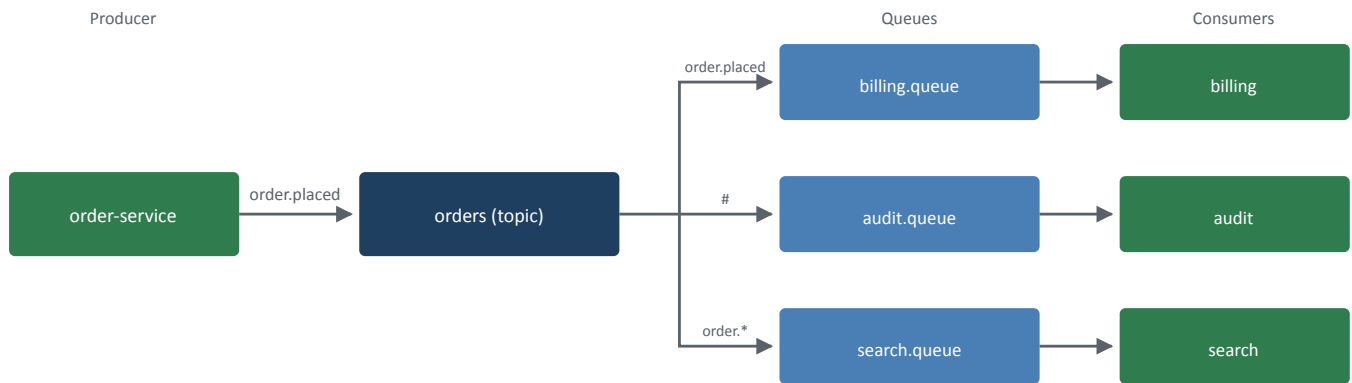
State	Means
Ready	Waiting for a consumer to take it
Unacked	Delivered, and not yet acknowledged
Total	Both of the above

An unacknowledged message is still in the queue. It is hidden from other consumers rather than removed, so a consumer that dies without acknowledging returns every message it held to ready. Those three numbers are the first thing to read in the management interface.

10. Exchange

An exchange receives messages from producers and decides which queues get them. Its type decides how, and section 11 covers the four.

The exchange is the only thing a producer names. Which queues exist, and which of them want a particular message, are decisions the consuming side made when it declared its bindings, and the producer never learns any of it.



The routing key is on the message; the binding keys are on the links. The exchange compares them and copies the message to whatever matches.

The exchange does not store the message, it copies it. A message matching three bindings becomes three messages in three queues, and each is acknowledged separately.

11. Exchange types

Direct: the routing key must equal the binding key.

```

logs (direct)
  error.queue ← "error"
  info.queue  ← "info"

routing key "error" → error.queue
routing key "warn"  → discarded, nothing is bound to it
  
```

Fanout: the routing key is ignored and every bound queue gets a copy.

```

order-events (fanout)
  billing.queue, audit.queue, search.queue

any routing key → all three queues
  
```

Topic: the routing key is words, and the binding key is a pattern.

```

orders (topic)
  eu.queue ← "order.*.eu"
  audit.queue ← "#"

routing key "order.placed.eu" → eu.queue and audit.queue
routing key "order.placed.us" → audit.queue only
  
```

Headers: matched on headers, and the routing key is ignored entirely.

binding arguments:

x-match: all every listed header must match; "any" needs only one
format: pdf
type: report

Type	Use it when
<code>direct</code>	The category is a single fixed value
<code>fanout</code>	Everyone bound should get everything
<code>topic</code>	Consumers want different slices of the same stream
<code>headers</code>	Routing depends on more than one attribute

A topic exchange can do everything the other two can. A topic binding with no wildcards behaves as direct, and a binding of `#` behaves as fanout. Direct and fanout exist because they are faster, not because they can do anything topic cannot.

12. Exchange against queue

	Exchange	Queue
Stores messages	No	Yes. It is the only thing that does
Producers address	Yes, always	Never
Consumers read from	No	Yes
Has a type	Yes, four of them	No
Decides routing	Yes	No
Survives a restart	If declared durable	If declared durable

The exchange is a router with no memory, and the queue is memory with no routing. Every question about RabbitMQ's behavior resolves to which of those two is responsible.

13. Binding and binding key

A binding is the link between an exchange and a queue, carrying a binding key.

```
BindingBuilder.bind(billingQueue)
    .to(ordersExchange)
    .with("order.placed");
```

No binding means the message is discarded. An exchange with nothing bound to it accepts publishes and throws them away, which is the usual reason a message "disappears" with no error anywhere.

A queue can have several bindings. Binding `billing.queue` to both `order.placed` and `order.cancelled` puts both kinds of message in one queue.

	Set by	Lives on	Changes
Routing key	The producer	Each message	Per message
Binding key	The consuming side	The binding	Rarely

Exchange-to-queue routing is five steps, and naming them answers most questions about where a message went:

- 1 the producer publishes to an exchange, with a routing key
- 2 the exchange looks at every binding it has
- 3 the exchange type decides whether each binding key matches
- 4 every matching queue receives its own copy
- 5 no match at all: discarded, or returned if published as mandatory

An exchange can also be bound to another exchange. The same rules apply, and it is how a common prefix is routed once and then split further.

14. Routing key

The routing key is a string the producer attaches to the message. What the exchange does with it depends on the exchange type.

Two wildcards are available in a topic binding, and they behave differently:

Wildcard	Matches
*	Exactly one word
#	Zero or more words

routing key: `order.placed.eu`

<code>order.placed.eu</code>	matches
<code>order.*.eu</code>	matches
<code>order.#</code>	matches
<code>order.*</code>	does not. The key has three words, not two
<code>#</code>	matches everything

The routing key belongs to the message and the binding key to the link. They are compared, never assigned to each other, and mixing up which is which is the usual confusion.

Order the words from general to specific. `order.placed.eu` works and `eu.placed.order` does not, because both wildcards match from the left and consumers bind on the shared prefix. Getting the hierarchy backwards means every binding needs a leading `#` and the exchange stops being selective.

15. Acknowledgement and rejection

An acknowledgement tells the broker a message was handled and can be removed from the queue.

Call	Means	The message
<code>ack</code>	Handled	Removed
<code>nack</code> with <code>requeue true</code>	Failed, try again	Back to ready
<code>nack</code> with <code>requeue false</code>	Failed, give up	Dead-lettered, or dropped

Manual acknowledgement is the only safe mode for work that matters.

	Automatic	Manual
Acknowledged	The moment it is delivered	When the code says so
A crash while processing	The message is lost	It returns to ready
Suits	Metrics, and nothing else	Everything else

```
@RabbitListener(queues = "billing.queue")
public void handle(OrderPlaced event) {
    charge(event);    // an exception here nacks the message
}
```

Spring's `AUTO` mode is not AMQP's auto-acknowledge, despite the name. Spring acknowledges after the method returns and nacks when it throws, which is the safe behavior. AMQP's `autoAck` acknowledges on delivery and loses the message on a crash. The two share a word and mean opposite things.

Requeueing a message that always fails is an infinite loop. It returns to the queue, is redelivered, fails again, forever. Use `requeue=false` with a dead-letter exchange, or a retry policy with a maximum number of attempts.

A consumer that never acknowledges quietly stops working. Unacknowledged messages count against the prefetch limit, so once it holds that many the broker sends it nothing more. The consumer looks connected and idle, and the queue grows behind it.

16. Message delivery semantics

Guarantee	Means	How it is reached
At most once	Never duplicated, and can be lost	Automatic acknowledgement
At least once	Never lost, and can be duplicated	Manual acknowledgement, durable, persistent
Exactly once	Delivered precisely once	The broker does not offer it

RabbitMQ gives at-least-once, and that is what to say. The reason exactly-once is impossible at the broker is simple: an acknowledgement can be lost after the work is done, and from the server's side that is indistinguishable from a consumer that died before doing it. Faced with the two, the broker delivers again.

Not losing a message and not repeating its effect are two different jobs on two different sides:

To not lose it	To not repeat its effect
A durable queue	An idempotent consumer
<code>delivery_mode: 2</code> on the message	Recording what has been handled
Manual acknowledgement	A unique constraint on the business key
Publisher confirms	—

Publisher confirms close the gap before the queue. Without them a broker restart between the publish and the write loses a message the producer believes it sent, and no amount of consumer-side care recovers it.

The `redelivered` flag is a hint, not proof. RabbitMQ sets it when a message is being delivered again, which is useful for logging. It can also be true on a first delivery after a restart, so it must never be the only thing preventing duplicate work.

17. Consumer concurrency

A consumer handles several messages at once in one of two ways, and both are usually in play.

```
spring.rabbitmq.listener.simple.concurrency=5
spring.rabbitmq.listener.simple.max-concurrency=20
spring.rabbitmq.listener.simple.prefetch=10
```

Setting	Controls
<code>concurrency</code>	Threads started for this listener
<code>max-concurrency</code>	How far it may scale up under load
<code>prefetch</code>	Unacknowledged messages held per thread

In-flight messages are concurrency multiplied by prefetch. Five threads at a prefetch of ten is fifty messages held by one instance, and ten instances of that is five hundred. It is the number that decides memory use and downstream load, and it is the one nobody calculates.

Each consumer thread gets its own channel, and channels are not thread-safe. Sharing one across threads corrupts the protocol in ways that surface as confusing errors much later. The framework handles this; hand-written consumers frequently do not.

Concurrency and ordering cannot both be had. One queue with one consumer processes in order. Add threads and messages are handled at the same time and finish in any order. Where order matters the options are a single consumer, or a consistent-hash exchange that sends related messages to the same queue so each queue stays single-threaded.

18. Retry mechanism

There are three ways to retry, and they are not equally good.

Local retry repeats the handler in process. Nothing returns to the broker, so it is fast and simple.

```
spring.rabbitmq.listener.simple.retry.enabled=true
spring.rabbitmq.listener.simple.retry.max-attempts=3
spring.rabbitmq.listener.simple.retry.initial-interval=1000
spring.rabbitmq.listener.simple.retry.multiplier=2
```

It holds the thread while it waits, and a crash loses the attempt, so it suits short failures rather than an outage.

Requeueing sends it back to the broker, immediately and forever. A `nack` with requeue returns the message to the front of the queue, where it is redelivered at once and fails again. It is the wrong answer on its own.

A delayed retry queue is the idiomatic pattern, built from a dead-letter exchange and a message TTL pointing at each other:

```
work.queue    x-dead-letter-exchange: retry.exchange
retry.queue   x-message-ttl:          30000
              x-dead-letter-exchange: work.exchange
```

A rejected message dead-letters into `retry.queue`, waits there for the TTL with no consumer attached, and expires. Expiring dead-letters it back to `work.exchange`, which delivers it to the work queue again, thirty seconds later.

Approach	Delay	Survives a crash	Complexity
Local retry	In process	No	Low
Requeue	None	Yes	Low, and usually wrong
TTL and dead-letter	Whatever the TTL is	Yes	Moderate
Delayed message plugin	Per message	Yes	Moderate

Attempts are counted in the `x-death` header. RabbitMQ records every dead-lettering there with a count, so the consumer can read it, and route a message past some threshold to a parking queue instead of retrying forever.

Something has to look at the parking queue. A message that failed every attempt needs a person, so an alert on that queue being non-empty is part of the design rather than an addition to it.

19. Virtual host

A virtual host is a namespace inside one broker. It has its own exchanges, queues, bindings, users and permissions.

```
/           the default vhost
/staging    its own exchanges and queues, same server
```

Names do not collide across vhosts. Two vhosts can each hold a queue called `orders`, and they are unrelated.

A vhost is an administrative boundary, not a performance one. They share the broker's memory, disk and Erlang processes, so a runaway queue in one affects the others. Separating environments that matter means separating brokers.

20. RabbitMQ vs a REST API

	REST API	RabbitMQ
The caller blocks	Until the response arrives	Only until the broker accepts
Knows the receiver	By address	Not at all. It knows an exchange
Receivers per call	One	However many queues are bound
The receiver is down	An error, immediately	Nothing happens. It stays queued
Failure handling	The caller retries	The broker redelivers

REST is right when the next line of code uses the result. A queue there only adds a way to wait for something HTTP already returns.

RabbitMQ is right when the work belongs after the reply. Charging a card, writing an audit row and updating an index are all things the customer should not be held for.

A reply is possible and usually a sign REST was wanted. The `reply_to` and `correlation_id` properties support request-reply over the broker, and it adds a queue, a correlation and a timeout to something an HTTP call does in one line.

The pieces together

Piece	What it is
Producer	Publishes to an exchange, never to a queue
Exchange	Decides which queues receive a copy
Binding	The link, carrying a binding key
Routing key	On the message, compared against binding keys
Queue	The only thing that stores messages
Consumer	Takes from a queue and acknowledges
Prefetch	Why work is shared, or is not
Ready and unacked	Where a message is before it is finished
<code>ack</code> and <code>nack</code>	Removed, requeued, or dead-lettered
Exchange type	Exact, everything, pattern, or headers
At least once	What the broker promises, and what it does not
concurrency × prefetch	How many messages one instance really holds
TTL and dead-letter	How a delayed retry is built
<code>delivery_mode</code> and <code>durable</code>	Both needed for a message to survive a restart
Vhost	A namespace, not an isolation boundary
Queue count	Which of the two patterns is actually in use

For example, take a placed order. The order service publishes once to the `orders` topic exchange with routing key `order.placed`, and returns to the customer. Three queues are bound with `order.placed`, `#` and `order.*`, so all three receive a copy and the billing, audit and search services each handle their own. The search service happens to be restarting for a deploy, so its queue depth climbs for a minute and drains once it

is back. Billing runs three instances on one queue with a prefetch of twenty, so the work is spread rather than duplicated. A message that keeps failing is dead-lettered after three attempts and somebody is paged about the dead letter queue.

The sentence worth carrying is that the producer knows an exchange and nothing else. Every question about who receives what is answered by the bindings, and those belong to the consumers.